

App Store & Play Store Readiness Report

A full pre-submission audit of Edgecipline — Next.js 16 static export wrapped in Capacitor 6 — against Google Play Developer Program Policy and the Apple App Store Review Guidelines.

GOOGLE PLAY

Not ready

Six policy and completeness blockers. A fake payment form ships inside the release APK, there is no account-deletion path, and the bundle points at the staging API.

APPLE APP STORE

Cannot submit

Everything above, plus the iOS target has never been configured or compiled. Sign-in, camera and push are all broken on iOS today.

11

P0 BLOCKERS

10

P1 HIGH

8

P2 MEDIUM

~1w

TO ANDROID

~3w

TO IOS

Verdict

Scope: Android (Google Play) and iOS (Apple App Store) release readiness, covering build configuration, store policy compliance, native platform setup, and console submission prerequisites.

STORE	STATUS	REASON
Google Play	NOT READY will be rejected	Six policy/completeness blockers. A fake payment form ships inside the release APK, there is no account-deletion path, and the bundle points at the staging API.
Apple App Store	CANNOT SUBMIT	Everything above, plus the iOS target has never been configured: no Firebase config, no privacy manifest, no usage-description strings, no push entitlement. Sign-in, camera and push are all broken on iOS today.

Realistic effort to a submittable build: Android ≈ 1 week · iOS ≈ 2–3 weeks. iOS also requires a Mac and an Apple Developer account before anything can be built at all — the project has never been compiled.

What is already right

Credit where it is due — a large share of the hard release engineering is done. The problems in this report are almost entirely product and compliance gaps, not architecture gaps.

- ✓ Release signing is env / `keystore.properties` driven and **fails the build** if unconfigured

✓ `targetSdk 36` / `compileSdk 36` — meets Play's 2026 target-API requirement

✓ `android:allowBackup="false"`

✓ Sentry wired for crash reporting, with `send-default-pii=false`

✓ Genuine in-app Privacy Policy and Terms with proper trading + AI disclaimers

✓ Notification icon, accent colour and channels correctly declared
- ✓ R8 minification and resource shrinking enabled on release

✓ Cleartext traffic disabled via `network_security_config.xml`

✓ Release output is an **AAB** (`bundleRelease`), not an APK

✓ Build-time environment validation that hard-fails on bad config

✓ Adaptive launcher icons at all densities, splash at all densities

✓ Secrets (`.env` , `keystore.properties` , `*.keystore`) correctly git-ignored

Blockers

Each of these causes a guaranteed store rejection or a functionally broken app. All eleven must close before either submission.

P0-1 No way to delete an account — violates both stores

There is **no account-deletion feature anywhere** in the product. Verified: zero matches across `frontend/app`, `frontend/components`, `frontend/services`, and no delete-account route in `backend/routes` — the only `router.delete` handlers cover individual trades.

- **PLAY • USER DATA** Any app allowing account creation must offer in-app deletion **and** a publicly reachable web deletion URL declared in Play Console → Data safety. `edgecipline.com/delete-account` currently returns **404**.
- **APPLE • 5.1.1(v)** Apps supporting account creation must let users initiate deletion **from within the app**.

Fix → Add `DELETE /api/profile/account` (hard-delete or anonymise User, Trades, Reflections, Checklists, device tokens, Cloudinary uploads and the Firebase Auth user), a confirm-flow entry in `app/profile/page.js`, and a public `/delete-account` web page describing the process.

P0-2 A fake payment form ships inside the release build

`components/SmartPaywall.js` is live and mounted in `app/upload-trade/page.js:317`. Its checkout loader branches on platform:

```
// SmartPaywall.js:26
function isSandboxCheckout() {
  return isNativeAndroidApp() || !String(process.env.NEXT_PUBLIC_RAZORPAY_KEY_ID || "").trim();
}
```

Two separate failures fall out of that one line:

On Android — the check is `true`, so the app injects `utils/mockRazorpay.js`: a full-screen **imitation Razorpay checkout** with card-number, expiry and CVV inputs, a "Pay" button and a "simulate failure" button. It grants a paid subscription with no transaction. Confirmed present in the shipped bundle. **PLAY • DECEPTIVE BEHAVIOR + PAYMENTS**
Rejection, with app-suspension risk.

On iOS — the platform check is `false` and the Razorpay key is non-empty, so the sandbox branch is skipped and the app loads the **real** `checkout.razorpay.com/v1/checkout.js`. **APPLE • 3.1.1** Digital subscriptions must use In-App Purchase; a third-party processor is an automatic rejection. It would also fail at runtime — the key is a placeholder.

Fix → Pick one strategy; do not ship the current hybrid. **(a) Recommended for v1:** remove the paywall from both mobile builds. Sell on the web only and have the app read subscription status — Apple 3.1.3(b) *Multiplatform* permits honouring a subscription bought elsewhere, provided the app contains no purchase UI and no link to buy. **(b)** Implement Google Play Billing + StoreKit properly (weeks, not days). Either way, **delete** `utils/mockRazorpay.js` **from production builds entirely** — a fake payment screen must never be in a store artifact.

Blockers

P0-3 "Upgrade Plan" button leads to a 404 inside the app

`app/profile/page.js:349` renders a prominent gradient CTA:

```
{profile?.subscriptionStatus !== "active" && (
  <Link href="/pricing">Upgrade Plan </Link>
)}
```

There is **no** `/pricing` route — not in `frontend/app/`, and no `out/pricing*` in the static export. Every user whose subscription is not active — which includes **every fresh reviewer account** — taps this and gets a 404 screen.

APPLE • 2.1 APP COMPLETENESS PLAY • BROKEN FUNCTIONALITY Near-certain rejection on first review.

Fix → Remove the CTA (per PO-2a) or build the route. Then sweep for other dead links — `app/admin/users/page.js:49` also deep-links to `/pricing`.

P0-4 The release bundle is baked against the staging API

`frontend/.env.local` line 1 sets `NEXT_PUBLIC_API_URL=https://staging-api.stratedge.live/api`.

`output: "export"` compiles that string into the JS chunks — verified present in `out/`, `android/app/src/main/assets/public` and `ios/App/App/public`. `npm run android:build` reads the same file, so today's release AAB ships pointing at staging.

The guard meant to catch this has been widened to let it through:

```
// config/environment.js:5
const PRODUCTION_API_HOSTS = new Set(["api.stratedge.live", "staging-api.stratedge.live"]);
```

The comment three lines above literally reads *"Never point a release build here"* — but nothing enforces it.

Fix → Add a `.env.production` pinned to `api.stratedge.live`, and gate the staging host behind a non-production mode so a production build physically cannot resolve it.

P0-5 Placeholder Razorpay key passes production validation

`.env.local` sets `NEXT_PUBLIC_RAZORPAY_KEY_ID=rzp_live_placeholder`. The validator at `config/environment.js:127` only checks `startsWith("rzp_live_")`, and `hasPlaceholder()` at line 44 only matches `^(your_|replace_|example|change|<)`. The value sails through both.

So the "production requires a live key" control is defeated — and combined with PO-2, the iOS build would load a real Razorpay SDK holding a dead key.

Fix → Add `placeholder|dummy|test|sample` to `hasPlaceholder()` and enforce a minimum key length.

Blockers

P0-6 The full admin console ships inside the consumer app

Both mobile bundles contain the complete internal back-office:

```
android/app/src/main/assets/public/admin/{login, dashboard, users, payments,
trades, monitoring, feedback, issues, expired-users}
ios/App/App/public/admin/... (identical)
```

A reviewer — or any user — can navigate to `/admin/login` inside the app. `admin/users` exposes subscription activation/deactivation; `admin/payments` exposes revenue.

APPLE • 2.3.1 HIDDEN FEATURES Apple explicitly rejects for this. It is also an unnecessary attack surface — the auth check lives in `admin/layout.js`, which is client-side.

Fix → Exclude `app/admin/**` from the Capacitor export (separate build target, or a build-time route filter) and ship admin as a web-only deployment.

P0-7 iOS: Firebase is not configured — Google Sign-In is dead

- `ios/App/App/GoogleService-Info.plist` — missing
- `AppDelegate.swift` — never calls `FirebaseApp.configure()`
- `Info.plist` has no `CFBundleURLTypes` with the `REVERSED_CLIENT_ID` scheme

`@capacitor-firebase/authentication` is the only sign-in path with a configured provider (`google.com`). On iOS it will fail or crash on first tap.

Fix → Register the iOS app in the Firebase console, add `GoogleService-Info.plist` to the Xcode target, call `FirebaseApp.configure()`, and add the reversed-client-ID URL scheme.

P0-8 iOS: no privacy manifest (PrivacyInfo.xcprivacy)

Required by Apple since May 2024 for apps using "required reason" APIs. Capacitor core uses `UserDefaults`; the app also touches file timestamps and disk space. Submitting without it produces **ITMS-91053: Missing API declaration** and the build is rejected at upload.

Fix → Add `PrivacyInfo.xcprivacy` to the App target declaring at minimum `NSPrivacyAccessedAPICategoryUserDefaults` (reason `CA92.1`), plus your data-collection types: email, name, user content, crash data, device ID.

Blockers

P0-9 iOS: missing camera / photo usage descriptions → guaranteed crash

The app uploads trade screenshots — Android declares `CAMERA` and `READ_MEDIA_IMAGES` (`AndroidManifest.xml:91-97`). Inside a `WKWebView`, `<input type="file" accept="image/*">` triggers the iOS camera and photo picker.

`Info.plist` contains **no** `NSCameraUsageDescription` and **no** `NSPhotoLibraryUsageDescription`. iOS terminates the app immediately when a permission is requested without its string, and the binary is rejected at upload (**ITMS-90683**).

Fix → Add both keys with honest, specific copy, e.g. *"Edgecipline uses your camera to capture broker screenshots so trades can be logged automatically."*

P0-10 iOS: push notifications cannot work

`@capacitor/push-notifications` is a dependency and appears in `packageClassList`, but:

- there is **no** `App.entitlements` file → **no** `aps-environment` key → APNs registration fails and Xcode blocks the upload
- `AppDelegate.swift` does not forward the registration callbacks the Capacitor plugin requires
- no** APNs auth key uploaded to Firebase (required for FCM → APNs)

```
func application(_ application: UIApplication,
  didRegisterForRemoteNotificationsWithDeviceToken deviceToken: Data) {
  NotificationCenter.default.post(name: .capacitorDidRegisterForRemoteNotifications, object: deviceToken)
}
func application(_ application: UIApplication,
  didFailToRegisterForRemoteNotificationsWithError error: Error) {
  NotificationCenter.default.post(name: .capacitorDidFailToRegisterForRemoteNotifications, object: error)
}
```

P0-11 The public Privacy Policy URL is broken

Both consoles require a live, publicly reachable privacy policy URL, and Play flags the listing if it ever stops resolving.

URL	STATUS
<code>https://www.edgecipline.com/privacy-policy</code>	404
<code>https://www.edgecipline.com/privacy</code>	200
<code>https://www.edgecipline.com/terms</code>	200
<code>https://www.edgecipline.com/support</code>	404

The repo defines `app/privacy-policy/` and has **no** `app/privacy/` — so the live site is a **stale deploy**. The moment you redeploy the current tree, `/privacy` starts 404-ing and the URL registered with Google and Apple dies.

Fix → Redeploy the site from this tree, settle on one canonical path, add a redirect from the other, and ship `/support` and `/delete-account` publicly before submitting.

High — rejection risk, or breaks on launch day

These will not necessarily stop the first review, but each one either invites a policy question or silently breaks the app in production.

P1-1 Signing key is compromised, and the artifacts are still tracked

The June report claimed these were "staged for untracking." They remain tracked today: `frontend/android.zip` (21 MB) and `frontend/android/app/build/outputs/apk/debug/app-debug.apk`.

`android.zip` contains `google-services.json`, `local.properties`, a debug APK and the **old plaintext signing properties**, and is still in git history (commit `6beecdd`).

Fix → `git rm --cached` both, rotate the upload key, then `git filter-repo` before the repo is ever shared.

P1-2 Google Sign-In will break the day you publish

`google-services.json` registers exactly **one** Android OAuth client (cert hash `2c2e3202...`). Play App Signing **re-signs your AAB with Google's key**, so the installed app's SHA-1 will not match anything in Firebase.

Fix → After the first upload, copy the *App signing key certificate* SHA-1 from Play Console → Setup → App integrity into Firebase, re-download `google-services.json`, and rebuild. Add the upload-key SHA-1 too.

ID	ISSUE	DETAIL AND FIX
P1-3	SCHEDULE_EXACT_ALARM needs a Play declaration	<code>AndroidManifest.xml:108</code> . On API 34+ this is revoked by default, and Play restricts it to alarm-clock/calendar apps via a sensitive-permission form. A trading-checklist reminder is unlikely to qualify. Drop it and rely on the WorkManager fallback already implemented.
P1-4	READ_MEDIA_IMAGES needs the Photo & Video declaration	<code>AndroidManifest.xml:95</code> . Play requires broad photo access to be core functionality, otherwise you must use the Android Photo Picker. Also missing <code>READ_MEDIA_VISUAL_USER_SELECTED</code> for Android 14 partial access.
P1-5	FOREGROUND_SERVICE declared with no service	<code>AndroidManifest.xml:112</code> — no <code><service></code> exists. On API 34+ every FGS needs a <code>foregroundServiceType</code> and a matching granular permission. Unused sensitive permissions attract review questions. Delete the line.
P1-6	versionCode 1 is hard-coded	<code>build.gradle:44</code> . Every Play upload needs a strictly higher code; you will hit "version code already used" on the second submission. Drive it from CI.

High — rejection risk, or breaks on launch day

ID	ISSUE	DETAIL AND FIX
P1-7	iOS project is unbuilt and misconfigured	<code>IPHONEOS_DEPLOYMENT_TARGET = 13.0</code> — below Xcode 16's floor, raise to 15.0+ · <code>CODE_SIGN_IDENTITY = "iPhone Developer"</code> — deprecated, release needs <i>Apple Distribution</i> · <code>DEVELOPMENT_TEAM</code> unset · no <code>Podfile.lock</code> and no <code>Pods/</code> → pod install has never been run · <code>UIRequiredDeviceCapabilities = [armv7]</code> in <code>Info.plist:29-31</code> is 32-bit, unsupported since iOS 11 → change to <code>arm64</code> · no <code>ITSAppUsesNonExemptEncryption</code> key, so App Store Connect blocks every submission on the export-compliance question.
P1-8	Notification permission requested cold at first launch	<code>MainActivity.java:37</code> calls <code>requestNotificationPermissionIfNeeded()</code> inside <code>onCreate()</code>, before the user has seen anything. Not a hard rejection, but it tanks opt-in rates and Apple reviewers dislike it. Ask after onboarding, with a rationale screen.
P1-9	Verify Apple Guideline 4.8 (Sign in with Apple)	Google is the only third-party provider. Because email/password registration also exists, 4.8 is <i>likelly</i> satisfied — but only if that path collects nothing beyond name + email and does no tracking. Confirm before submitting; adding Sign in with Apple removes all doubt.
P1-10	Reviewers land straight in a broken state	A brand-new reviewer account has <code>subscriptionStatus</code> <code>≡</code> "active" → sees the dead Upgrade button (PO-3) and, on upload, the mock paywall (PO-2). You must also supply Apple with a demo account in App Review notes — <code>app/verify-otp/</code> implies email verification, and reviewers cannot receive your OTP emails.

Medium — quality, polish, long-term cost

ID	ISSUE	DETAIL
P2-1	Google Fonts loaded from CDN in 29 files	<code>app/layout.tsx:45-49</code> plus 28 pages fetch <code>fonts.googleapis.com</code> at runtime. In a packaged app that means visible FOUT, a broken look offline, and GDPR exposure — German courts have ruled CDN font loading transmits IP without consent. Self-host via <code>next/font/local</code> .
P2-2	App ID typo is permanent	<code>com.edgecpline</code> vs the brand "Edgecipline" (<code>capacitor.config.ts:4</code>). Cannot be changed after first publication on either store. It is also only two segments — Apple convention is reverse-DNS (<code>com.edgecipline.app</code>). Last chance to fix is before the first upload.
P2-3	PWA manifest icons are fake	<code>public/manifest.webmanifest</code> declares one <code>logo.png</code> as 192×192, 512×512 and 180×180. Generate real sizes.
P2-4	Dev leftovers committed	<code>tmp-refactor.js</code> , <code>tmp-refactor-add.js</code> , <code>tmp-refactor-login.js</code> , <code>tmp-refactor-register.js</code> , <code>frontend/image.png</code> , root <code>image.png</code> .
P2-5	ESLint lints the build output	<code>globalIgnores</code> in <code>eslint.config.mjs</code> overrides the defaults and drops <code>ios/App/App/public/**</code> and <code>android/**/assets/public/**</code> , so lint reports 44,737 problems — roughly 44,300 from minified bundles. Real source issues are buried. Add those paths back.
P2-6	Privacy policy names the wrong product	<code>privacy-policy/page.js:352</code> says "Edge Discipline", not "Edgecipline". Reviewers do read these.
P2-7	Policy dated April 17, 2026	Refresh before submission. Both stores check that the policy covers what the app actually collects: screenshots, trade data, Firebase auth, Sentry, FCM tokens, Cloudinary uploads.
P2-8	Debug logging in shipped native code	<code>MainActivity.java</code> logs cookie and permission state via <code>Log.d</code> . Harmless but noisy — gate on <code>BuildConfig.DEBUG</code> .

Store console work still outstanding

None of this exists yet, and none of it is code.

Google Play Console

- ☐ Developer account (\$25) + D-U-N-S / identity verification — can take 1–2 weeks
- ☐ 20 testers for 14 continuous days on closed testing before production access is granted to new personal accounts
- ☐ Data safety form — collection, sharing, encryption in transit, deletion URL
- ☐ Content rating questionnaire — expect finance / simulated-gambling probes
- ☐ Target audience & content (must be 18+)
- ☐ Ads declaration (none) · News declaration (no)
- ☐ Financial features declaration — argue journal, not advisory
- ☐ Listing: 2–8 phone screenshots, 1024×500 feature graphic, 512×512 icon, short + full description
- ☐ Privacy policy URL (see PO–11)

App Store Connect

- ☐ Apple Developer Program (\$99/yr) + a Mac — iOS cannot be built or uploaded from Windows
- ☐ App Privacy "nutrition label" questionnaire
- ☐ Age rating — 17+ recommended for financial content
- ☐ Screenshots: 6.9" and 6.5" iPhone required; 12.9" iPad if you claim iPad support
- ☐ Decide iPad support — `Info.plist` currently declares iPad orientations
- ☐ Demo account credentials in App Review notes
- ☐ Support URL (currently 404) and marketing URL
- ☐ Export compliance answer (see P1–7)

Recommended order of work

Sequenced so the longest calendar dependencies start on day one and code work runs in parallel behind them.

Start today, before any code: open the Play Console account, complete identity verification, and begin the 14-day / 20-tester closed test. It is the longest pole in the schedule and it is pure waiting — every day you delay it is a day added to launch.

WEEK 1 Unblock Android

1. Delete `mockRazorpay.js`; strip the paywall and `/pricing` CTA from mobile builds P0-2, P0-3
2. Build account deletion end to end: API, profile UI, public web page P0-1
3. `.env.production` with the prod API; remove staging from the prod allowlist; tighten `hasPlaceholder()` P0-4, P0-5
4. Exclude `app/admin/**` from the Capacitor export P0-6
5. Redeploy the website; fix `/privacy-policy`, `/support`, `/delete-account` P0-11
6. Untrack `android.zip` and `app-debug.apk`; rotate the upload key P1-1
7. Drop `SCHEDULE_EXACT_ALARM` and `FOREGROUND_SERVICE`; wire `versionCode` to CI P1-3, P1-5, P1-6

WEEK 2 Android submission + iOS setup

8. Add the Play App Signing SHA-1 to Firebase; verify Google Sign-In on a Play-signed internal build P1-2
9. Self-host fonts P2-1
10. Decide the `com.edgeline` question — **now or never** P2-2
11. On a Mac: `pod install`, Firebase iOS config, `PrivacyInfo.xcprivacy`, usage-description strings, push entitlement + AppDelegate callbacks, deployment target, `arm64`, signing P0-7 → P0-10, P1-7

WEEK 3 iOS QA and submission

12. Full device pass on iOS: sign-in, camera upload, push, offline, deep links
13. Screenshots, listing copy, demo account, App Privacy label

IN ONE LINE

The engineering foundation is solid, but the app is **not shippable in its current state** — a mock payment screen and a dead upgrade button would fail review on their own, account deletion is legally mandatory and entirely absent, the bundle points at staging, and the iOS target has never been configured or compiled. Fix the eleven POs and **Android is a realistic one-week submission; iOS needs a Mac and roughly three.**